

Table of contents

Introduction	3	Why DevEx is the key to scalable AI adoption	10
--------------	---	--	----

About the AI Collaboration Matrix	4	AI tools across the SDLC	15
-----------------------------------	---	--------------------------	----

Research insights	6	How Meta is transforming software testing with AI-powered bug hunters	18
-------------------	---	---	----

5 common pitfalls of AI adoption in software development	9	How Google uses AI to speed up code migrations by 50%	22
--	---	---	----

Introduction

What does AI in the SDLC look like beyond Copilot adoption? And how do you measure its impact? These questions are top of mind for every engineering leader right now, and that’s why we created the **AI Collaboration Matrix**: a practical framework to help teams visualize where they are on the journey from human-led to AI-led work.

The matrix plots your organization across two critical axes:



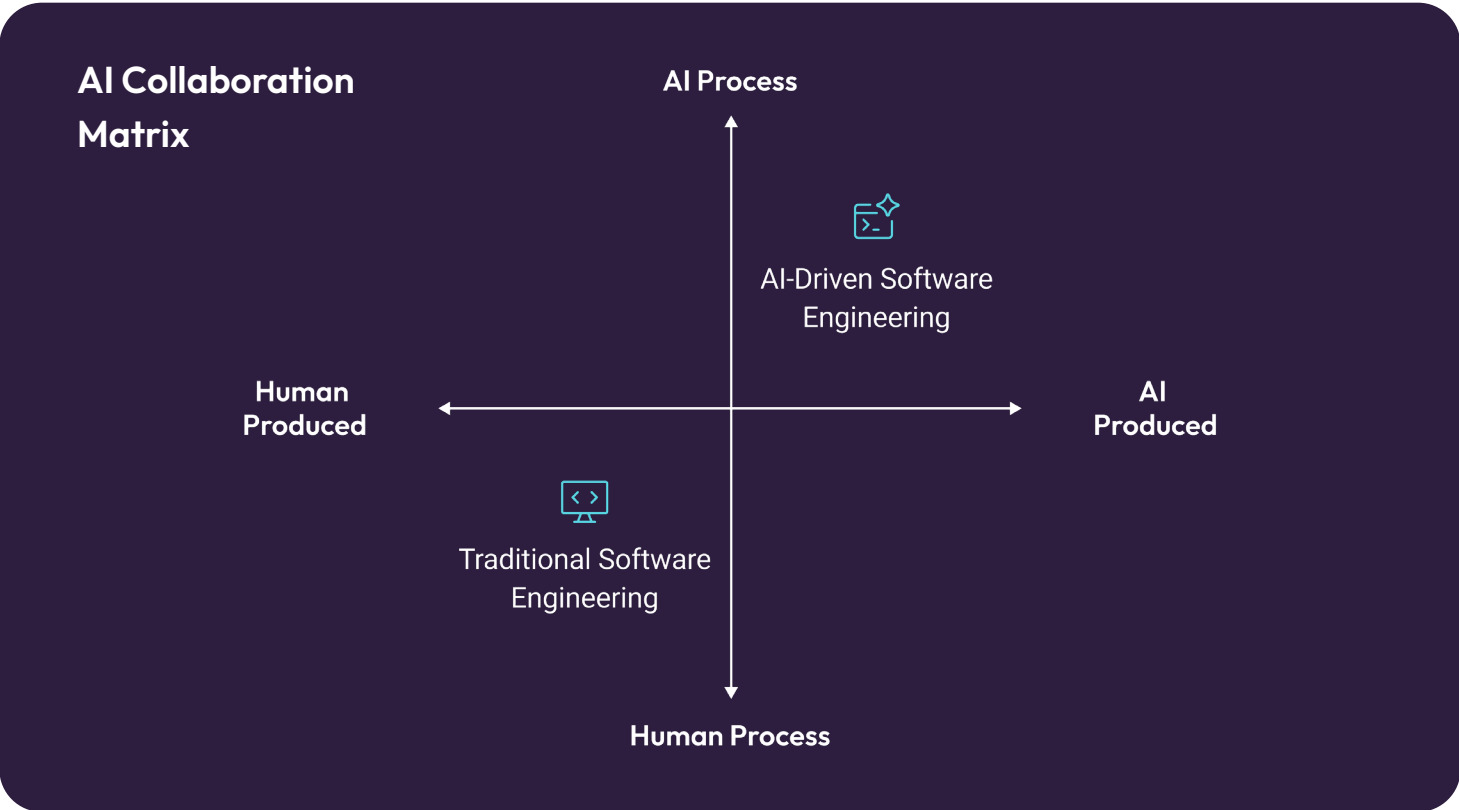
Who creates your software artifacts (Humans vs. AI).



Who drives your development workflows (again, Humans vs. AI).

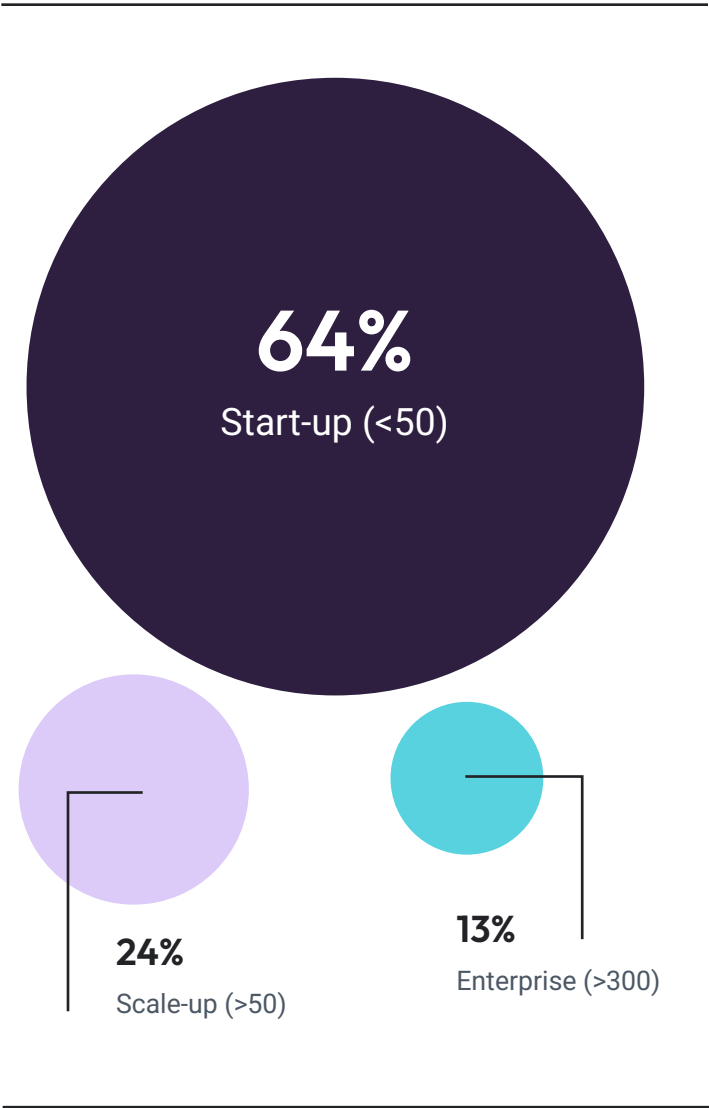
Most existing productivity frameworks – whether it’s SPACE, DORA, or others – were built for a world where humans did all the work. But now, AI is generating code, agents are coordinating workflows, and LLMs are embedded across the SDLC. The AI Collaboration Matrix provides a way for engineering teams to map and measure not just what’s being built, but who (or what) is building it.

This **DevEx Guide to AI-Driven Software Development** explores where over 400 engineering leaders fall on the AI Collaboration Matrix and what that tells us about the future of software development. We gathered data reaching a broad cross-section of leaders across company sizes, industries, and technical roles.



Here’s a snapshot of who participated:

- 63.8% came from Start-ups (<50 developers), offering insights into lean teams experimenting aggressively with AI.
- 23.6% represented Scale-ups (50–300 developers), typically navigating fast growth and tooling maturity.
- 12.6% came from Enterprises (>300 developers), where governance, security, and process orchestration take priority.



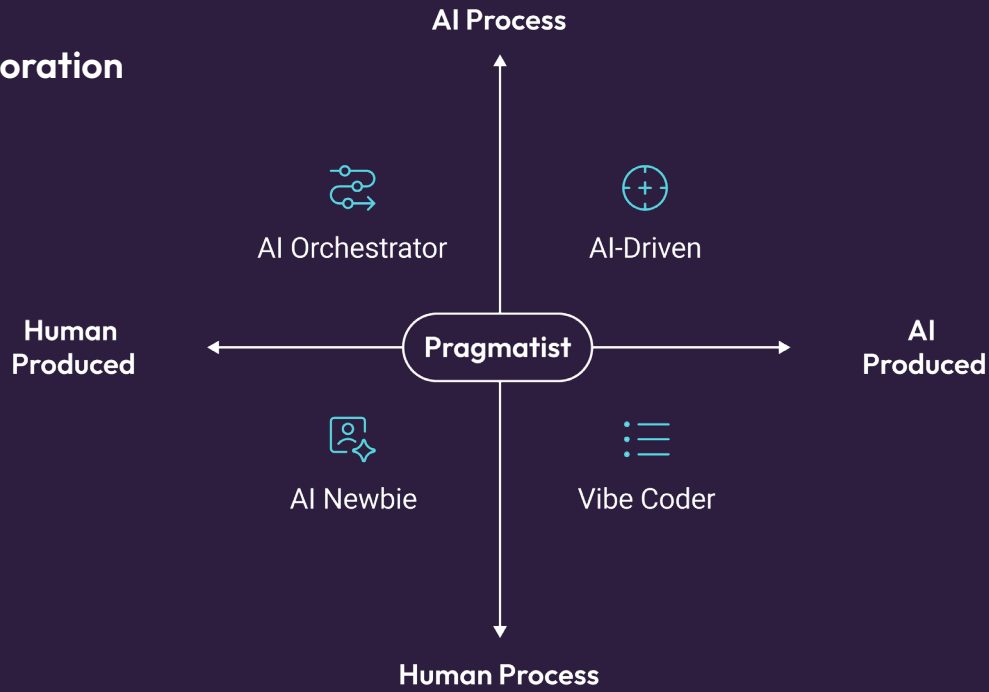
Across seven chapters, we’ll analyze survey results by role, tooling, and use case; explore common misconceptions; showcase emerging AI development patterns; and highlight how industry leaders like Meta and Google are applying AI today. Our goal is to help you navigate the AI shift with insight, not guesswork – and to give you the language and frameworks you need to lead your team into the future of AI-driven software development.

About the AI Collaboration Matrix

Understanding where your team or organization sits within the AI Collaboration Matrix provides crucial insights for planning the next steps in your AI adoption journey. Each quadrant presents distinct characteristics, opportunities, and challenges that require tailored strategies. When transitioning between quadrants, balance technical implementation with cultural adaptation. Different teams may occupy different quadrants simultaneously; leverage this diversity to accelerate learning across your organization.

Want to know where you fall on the matrix? [Take our quiz to find out.](#)

AI Collaboration Matrix



AI Newbie

This quadrant features minimal AI integration in content creation and workflow management, with high reliance on manual quality control and traditional knowledge transfer methods.



Vibe Coder

This approach leverages significant AI contribution to content/code generation while maintaining traditional human-led workflows for review and monitoring, resulting in increased output velocity but potential process bottlenecks.



AI Orchestrator

This quadrant combines strong human craftsmanship in content creation with sophisticated automation of workflow orchestration, enhancing consistency in delivery timelines while reducing coordination overhead.



AI-Driven

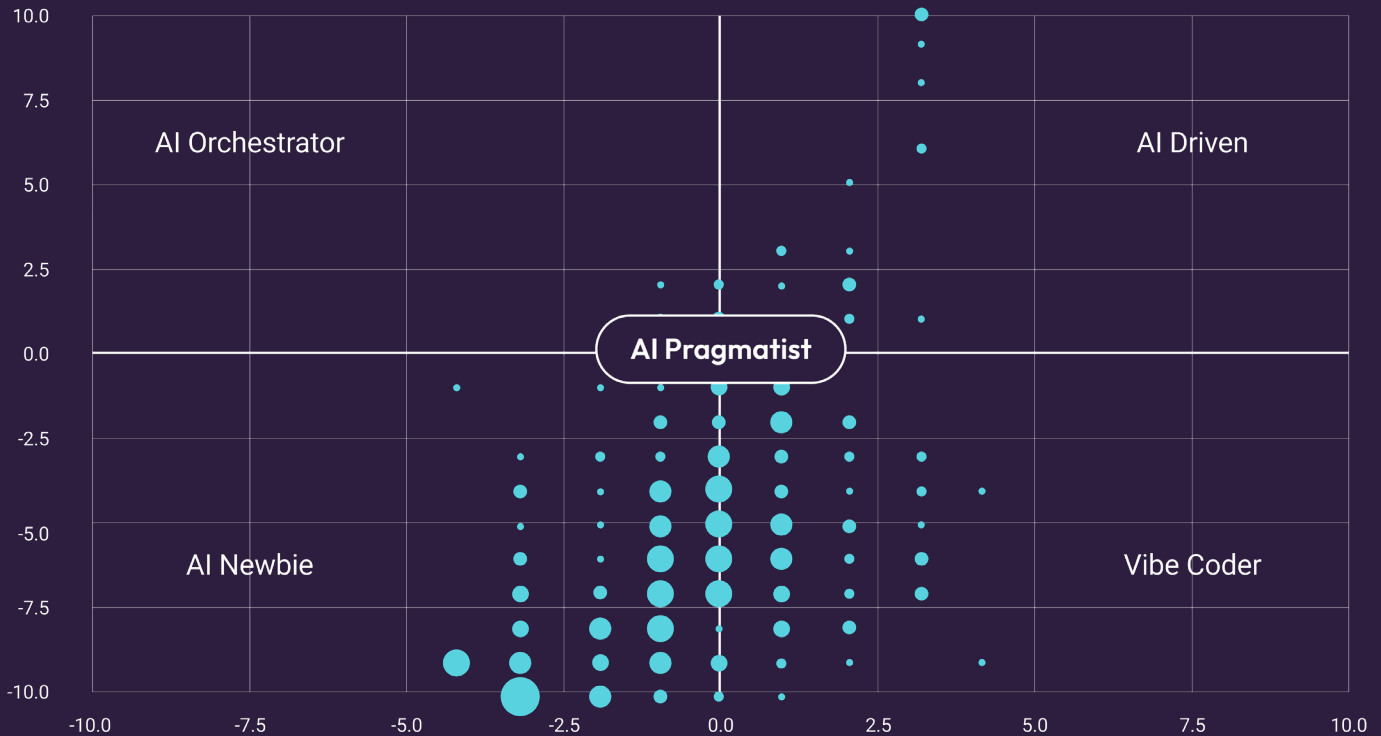
AI orchestrates content creation and workflow management while humans focus on strategic direction, enabling data-driven optimization that yields significant productivity and consistency gains.



AI Pragmatist

Balanced AI involvement across creation and workflows, with humans focused on strategy and exceptions, enabling continuous optimization that yields measured improvements in both quality and efficiency.

AI developer personas by vertical and horizontal scores



Key takeaway: Most teams are straddling the line between AI Newbie and Vibe Coder.

The largest cluster of respondents falls just below or around the center of the matrix. These teams are:

- Balancing human judgment with AI support.
- Experimenting with AI both in content/code generation and workflow automation.
- Likely engaging in incremental optimization, rather than radical transformation.

This placement indicates a majority of teams are “playing it safe”: they are adopting AI with measured steps rather than diving into full orchestration or complete AI-driven execution. These teams are also well-positioned to move into any of the more advanced quadrants, depending on their future priorities (e.g., deeper automation vs. more AI generated code).

Key takeaway: Few have reached the AI-Driven or AI-Orchestrator quadrants.

The top-right (AI-Driven) and top-left (AI Orchestrator) quadrants are sparsely populated, indicating that full AI integration (both in code creation and workflow orchestration) is still uncommon. Teams that do occupy these quadrants may be outliers: early adopters with strong internal AI infrastructure or aggressive automation goals. The low concentration here suggests that technical and cultural barriers remain significant, even among teams comfortable with GenAI tools.

In terms of company size relative to adoption, Enterprises remain largely risk-averse to AI. Out of the total, 50% of enterprise respondents are AI Newbies, and none are AI Orchestrators, while only 8% are AI Driven. Even in scale-ups, where 57% are AI Newbies and just 4% are AI Driven, the trend holds. The data reveals a consistent theme: AI is being used, but not yet embedded.

Key takeaway: Polarization between “Vibe Coders” and “AI Newbies” suggests fragmentation in adoption paths.

There are visible pockets in the bottom-left (AI Newbie) and bottom-right (Vibe Coder) quadrants. These groups reflect two divergent AI adoption trajectories:

- AI Newbies rely heavily on manual work with limited AI exposure.
- Vibe Coders employ heavy use of AI to generate content/code but have yet to evolve their processes to handle this shift.

This fragmentation implies that different teams (even within the same company) may experience AI adoption very differently depending on role, team culture, or leadership.

Among start-ups, 60% of respondents identify as AI Newbies, while 20% are AI Pragmatists and 16% are Vibe Coders. This reflects a high level of experimentation but limited structure. Only 1 start-up is categorized as an AI Orchestrator and just 7 are fully AI Driven – showing that most small teams are exploring GenAI without clear frameworks or repeatable practices.

Start-up survey results



60%

AI Newbies



20%

AI Pragmatists



16%

Vibe Coders

5 common pitfalls of AI adoption in software development

01

Using AI without context

Developers often rely on simple prompts with minimal project context, leading to low-quality, irrelevant, or incorrect AI-generated code.

04

No feedback or iteration loop

Teams often skip collecting usage data and feedback, thereby missing opportunities to refine prompts, tune models, and improve outcomes over time.

02

Lack of DevEx enablement

Without investment from Developer Experience teams (like prompt libraries, training, and integrated tooling) developers will struggle to use AI effectively and consistently.

05

Shallow experimentation without a strategy

Deploying AI as a trendy tool without aligning it to broader developer productivity and experience goals can result in low adoption and wasted potential.

03

Failure to integrate AI into workflows

When AI tools are siloed from IDEs, CI/CD systems, and repositories, they can't leverage essential information needed for meaningful suggestions.

Why DevEx is the key to scalable AI adoption

Amidst all the AI hype, a troubling trend is emerging: many teams are adopting AI in the most superficial way possible: through one-off tools and shortcut strategies that rely on simple prompts and minimal context.

At the heart of this problem is a widespread misunderstanding of how AI-driven development truly works. Most developers are handed an AI assistant or coding copilot and expected to “just prompt it.” The assumption is that developers can extract value by tossing in a few natural language commands and waiting for results. While this may work in isolated cases, it’s not quite a sustainable or scalable strategy.

The issue is that AI systems – especially large language models – are very context-sensitive. Without rich inputs like project structure, code history, architectural patterns, and team conventions, these tools operate in a vacuum. The result is code that might technically work, but lacks cohesion, introduces technical debt, or simply misses the mark. Worse, developers waste time reworking AI-generated suggestions, ultimately undermining any productivity gains.

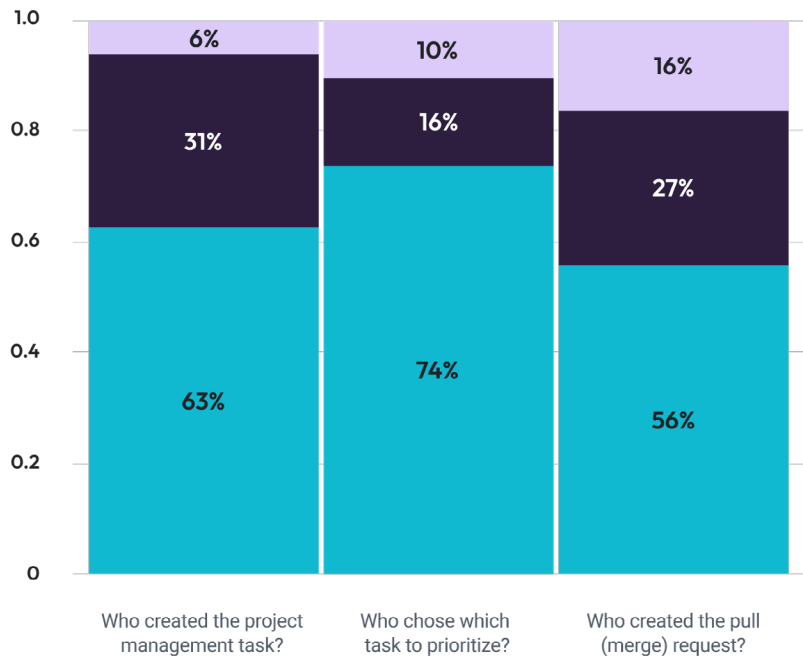
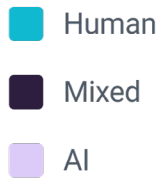
This shortcut mentality stems from treating AI like a passive tool rather than an active collaborator. Prompting alone is not a development methodology – it’s a surface-level interaction. For AI to become a true force multiplier, developers need support structures that provide the

necessary context and integration pathways. And this is where the Developer Experience (DevEx) team comes in.

DevEx plays a critical role in enabling AI-driven development. Their mission isn’t just to optimize pipelines and reduce friction; it’s to empower engineers with the right tooling, documentation, and processes. In the case of AI, this means going beyond rolling out a chatbot and calling it a day. Successful AI adoption requires DevEx to invest in foundational enablers: centralized prompt libraries, context-enriched APIs, training programs, integration with IDEs and repositories, and automated feedback loops that improve model responses over time. When DevEx teams build these scaffolds, they give developers the means to use AI with purpose.

To unlock the real promise of AI, organizations must go beyond surface-level prompting and invest in the infrastructure, processes, and enablement that make AI an integrated, intelligent part of the SDLC.

Project Planning



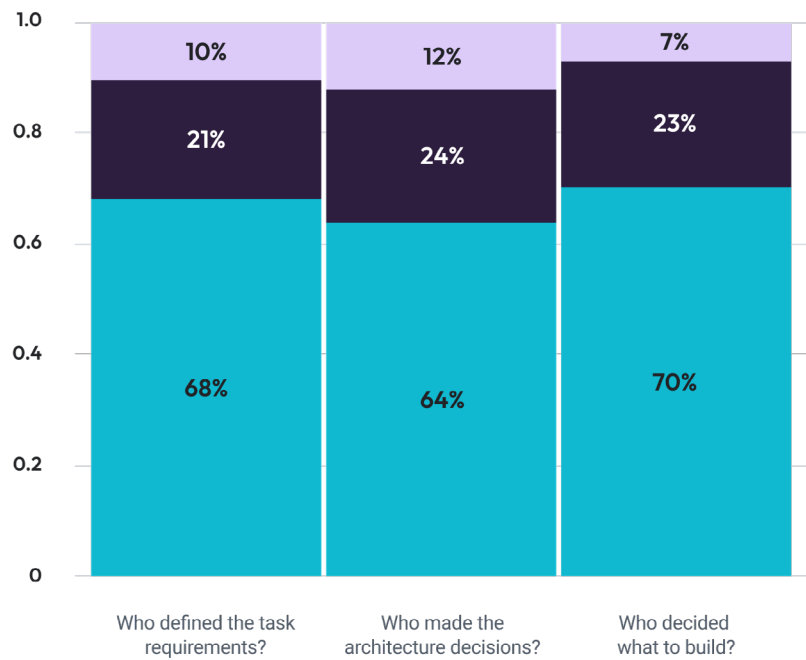
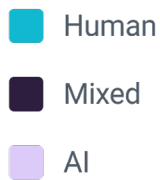
Key takeaway: Our research revealed that task creation is still largely a manual process, with 31% of teams using it to assist in task creation and 26% incorporating it into prioritization, though human oversight still dominates.

The highest AI adoption can be seen in the planning phase, where 43% of pull/merge requests now involve AI in some capacity. This signals a growing reliance on AI tools to streamline processes like structuring work and drafting PRs, while critical decisions (like prioritization) remain largely human-led.

Survey results

- Who created the project management task?
63% Human | 31% Mixed | 6% AI
- Who chose which task to prioritize?
74% Human | 16% Mixed | 10% AI
- Who created the pull (merge) request?
56% Human | 27% Mixed | 16% AI

Requirements & Architecture



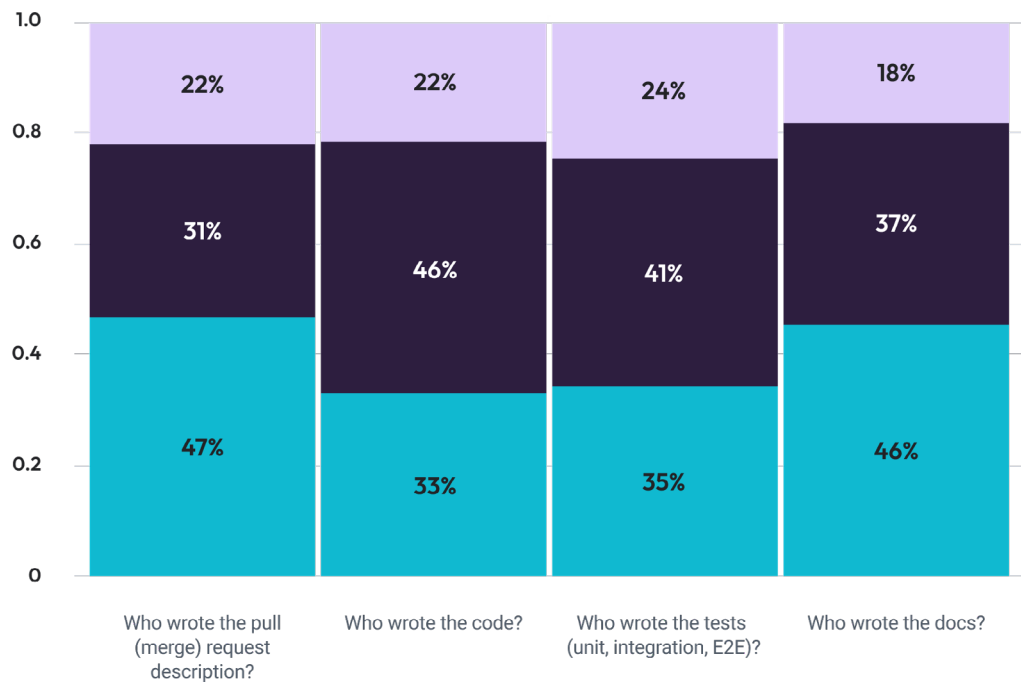
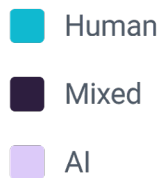
Key takeaway: While 68–70% of teams still rely on humans for these decisions, AI is trending up in support of architecture decisions, with a 36% participation rate in full or mixed involvement.

AI is beginning to support—but not replace—human judgment in critical product development areas like requirements gathering and feature planning.

Survey results

- Who defined the task requirements?
68% Human | 21% Mixed | 11% AI
- Who made the software architecture decisions?
64% Human | 24% Mixed | 12% AI
- Who decided what to build?
70% Human | 23% Mixed | 7% AI

Coding



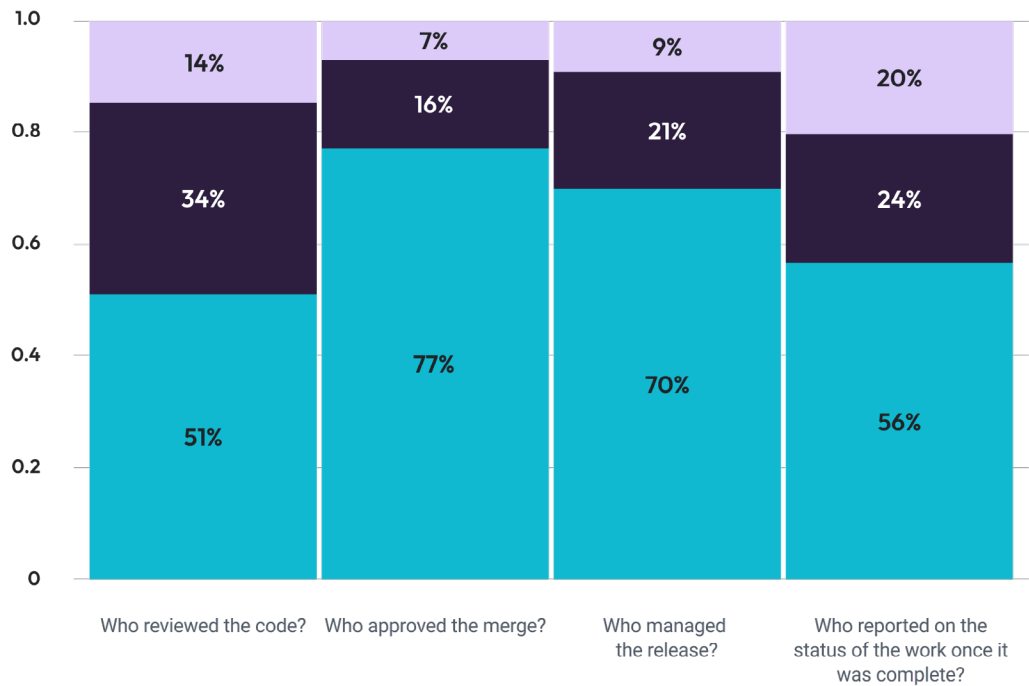
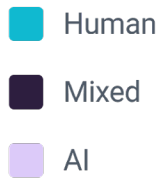
Key takeaway: AI is seeping into core development tasks quickly, with over half of teams using it to generate pull request descriptions and 67% relying on it to assist or fully write code.

Documentation and testing are also now increasingly automated, with 65% of teams using AI to support or handle these responsibilities entirely. These trends reflect a growing trust in AI for structured, repeatable tasks, freeing developers to focus more on higher-order problem solving and design.

Survey results

- Who wrote the pull (merge) request description?
47% Human | 31% Mixed | 22% AI
- Who wrote the code?
33% Human | 46% Mixed | 21% AI
- Who wrote the docs (e.g. integration, E2E)?
35% Human | 41% Mixed | 24% AI
- Who wrote the test or integration code (e.g.E2E)?
35% Human | 40% Mixed | 24% AI

Review & Release



Key takeaway: In the Review and Release segments of the SDLC, AI is still largely playing a supporting role.

Most teams still rely on manual reviews and approvals, though a growing minority (15–20%) are blending AI into these steps. However, “Human in the Loop” remains the standard for now.

Survey results

- Who reviewed the code?
51% Human | 34% Mixed | 15% AI
- Who approved the merge?
77% Human | 16% Mixed | 7% AI
- Who approved the code?
77% Human | 16% Mixed | 7% AI
- Who released the code?
70% Human | 20% Mixed | 10% AI

AI tools across the SDLC

These are the AI tools currently shaping the software development landscape. Each tool is mapped to the specific stage(s) of the SDLC it supports.



Code generation tools

- GitHub Copilot
- Amazon Q Developer
- Tabnine
- Windsurf



Code review & collaboration tools

- DeepCode
- Sider
- LinearB Workflow Automation
- Cursor



Testing tools

- Diffblue Cover
- Qodo
- Testim



Knowledge management & prioritization tools

- Stack Overflow for Teams (with AI)
- Jira with AI prioritization
- LinearB Workflow Automation
- Notion AI
- Roovo by Atlassian

Key takeaway: Enterprise teams are the most likely to use fully AI-driven workflows, especially in planning and review.

While start-ups and scale-ups generally show higher human ownership, enterprise organizations report the highest share of AI-only contributions across several SDLC areas:



Pull request creation

26% of enterprises report AI-only authorship vs. 21% in scale-ups and 13% in start-ups.



Code writing

AI-only usage sits at 26% in enterprises compared to 24% in start-ups and 21% in scale-ups.



Code review

8% AI-only in enterprises vs. 5% in start-ups and 11% in scale-ups.

This suggests that larger organizations are more likely to automate discrete engineering tasks through GenAI, perhaps due to their access to internal tooling, larger datasets, and the pressure to scale developer productivity.

Key takeaway: Start-ups lead in human + AI collaboration (“Mixed” usage), especially in planning and documentation.

In start-ups, mixed-mode usage (AI + human collaboration) is often higher than in scale-ups or enterprises across upstream activities:



Task prioritization

24% mixed in start-ups vs. 17% (scale-ups) and 24% (enterprises).



Architecture decisions

27% mixed in start-ups vs. 24% in both larger groups.



Test and integration code

34% mixed in start-ups vs. 35% (scale-ups) and 39% (enterprises).



Release management

23% mixed in start-ups vs. 30% (scale-ups) and 29% (enterprises).

This trend suggests that start-ups are embracing AI tools as collaborators rather than replacements, likely using copilots, prompt engineering, or assistants to move faster without ceding full control. Along that same vein, start-ups account for the highest absolute number of AI tool usage, with 42% reporting OpenAI, 27% Microsoft, and a long tail of smaller tools.

Key takeaway: Scale-ups appear to be in a transitional phase, with the most balanced distribution of Human, Mixed, and AI work.

Unlike start-ups (more human) or enterprises (more AI-heavy), scale-ups display a relatively even distribution across all three modes:



Code writing

35% human / 35% mixed / 30% AI



Test writing

38% human / 34% mixed / 27% AI



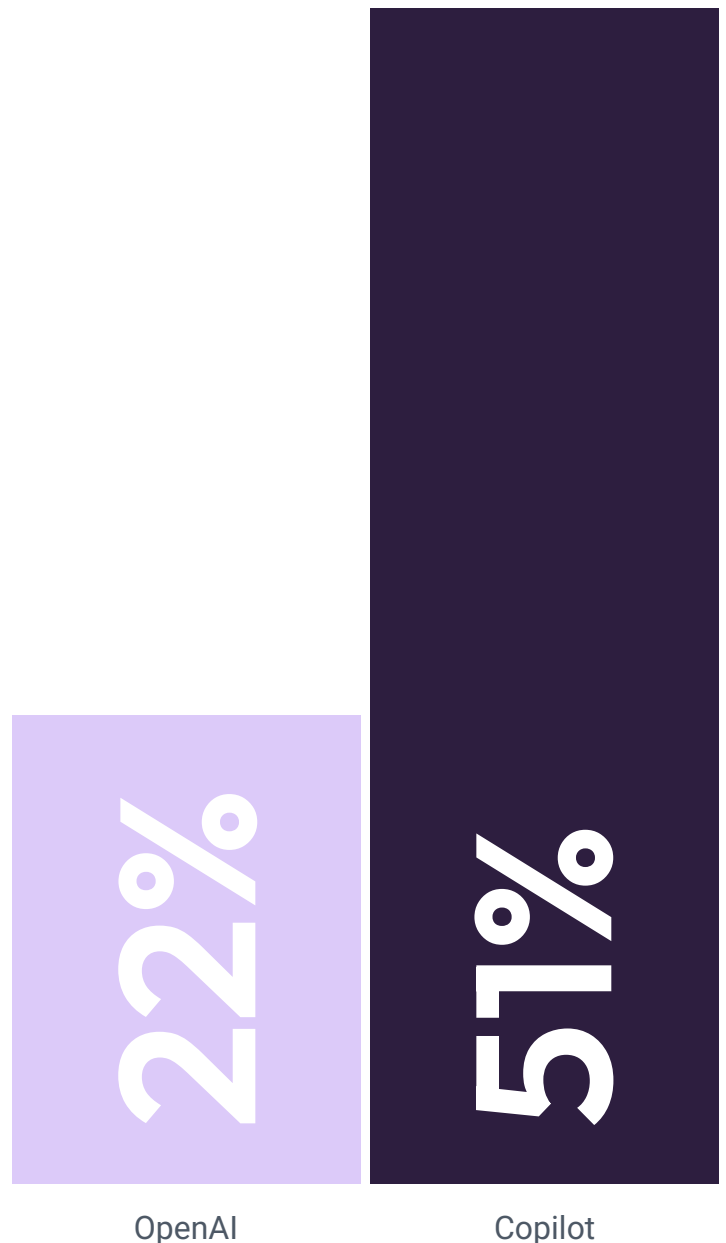
PR creation

45% human / 34% mixed / 21% AI

This reflects their unique position as part of the tipping point segment: scaling fast like start-ups but under growing pressure to standardize and systematize workflows like enterprises.

Many are piloting AI tools but haven't fully operationalized them yet. OpenAI (51%) and Microsoft (22%) dominate tool usage in scale-ups, making up the bulk of adoption. There is less diversity in the tools reported compared to start-ups, suggesting that scale-ups are standardizing their AI stack as they mature.

Dominant tools powering today's scale-ups



How Google uses AI to speed up code migrations by 50%

[A recent whitepaper from Google](#) offers fascinating insights into how the company is leveraging LLMs to automate code migrations at scale.

To summarize: AI wrote nearly 75% of the code, and Google developers believe it increased productivity by 50%. Let's look at the key findings and implications of this groundbreaking work.

Leveraging AI for routine system migrations

Google leveraged a relatively routine task for their experiment: transitioning identifiers from 32-bit to 64-bit integers. This functionally simple change requires modifications across potentially hundreds of files in multiple programming languages.

Traditional approaches to such migrations typically involve:

- 1 Manually testing changes to ensure correctness
- 2 Manually inspecting and modifying each reference
- 3 Using regular expressions to find potential code references
- 4 Managing the frustrating, time-consuming process of finding all instances requiring changes
- 5 When done manually, these migrations could take years to complete - as evidenced by a previous similar migration at Google that took approximately two years

To address these challenges, Google developed an automated system that integrates LLMs into the migration workflow. The system follows a structured process:

1. Find potential references

Using Google’s code indexing system (Kythe), the tool identifies direct references to the identifiers being migrated, as well as indirect references up to a distance of five.

2. Categorize references

The system classifies identified references into groups based on confidence levels:

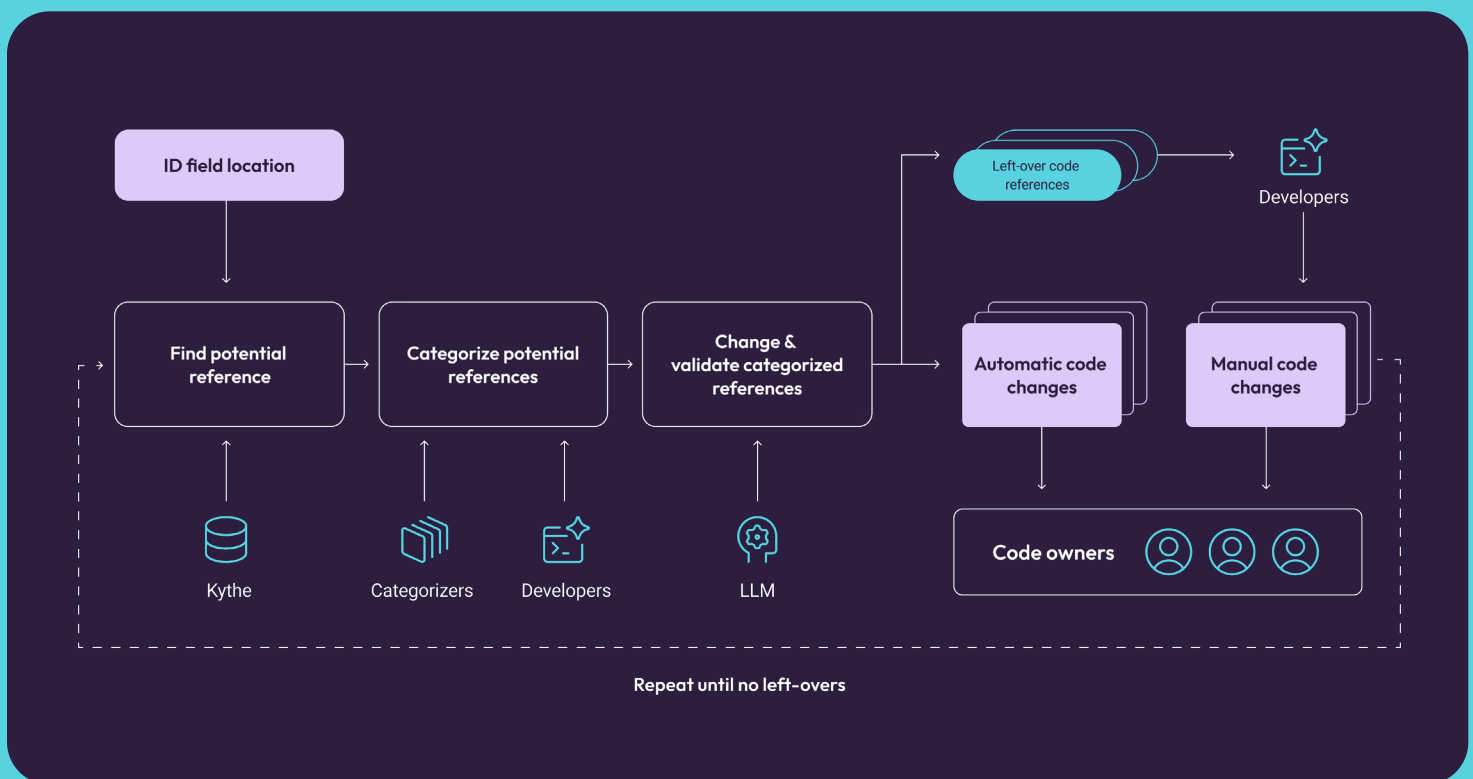
- **Not-migrated:** Locations identified with 100% confidence as not yet migrated
- **Irrelevant:** Locations determined to be irrelevant with 100% confidence

- **Relevant:** Locations that need further investigation
- **Left-over:** Locations intentionally set aside for manual developer review to determine if migration is required or to mark explicitly as irrelevant.

3. Change & validate

This is where the LLM comes into play. The system uses Google’s Gemini model (fine-tuned on internal Google code) to suggest appropriate code changes, which are then validated through multiple steps to ensure correctness.

The validation process is particularly thorough, including checks for successful completion, whitespace-only changes, AST parsing, a “punt” check where the LLM itself determines if the change was necessary, and build/test validations.



High level overview of the automated ID migration system: The system runs nightly, finds potential ID references and categorizes them. Developers make necessary code changes using an LLM or manually (if the LLM failed to make the change), and send the changes to code owners. This process continues until no more ID references are left over to migrate.

How AI handled most of the migration workload

The results of this approach are remarkable. Over twelve months, three developers used the system to perform 39 distinct migrations, resulting in:

- 595 total code changes submitted
- 93,574 total edits across all changes
- 69.46% of edits performed by the LLM
- 74.45% of code changes were generated by the LLM
- An estimated 50% reduction in developer time compared to manual approaches

Google's developers reported high satisfaction with the automated system, particularly appreciating how AI helped with continuous progress, system accuracy, and reducing tedious work.

Google's strategic AI rollout is key to their success

Despite its success, the system wasn't without challenges. Google reported the following issues:

- **LLM context window limitations:** Large files sometimes couldn't fit entirely within the LLM's context window, requiring manual intervention. Although newer versions of Gemini now have a much larger context window that may mitigate this issue.
- **LLM hallucinations:** The model occasionally produced irrelevant changes, unnecessary comments, or simply reformatted code without making meaningful changes.

- **Language support variances:** The LLM performed well with languages like Java, C++, and Python, but struggled with less common languages like Dart.
- **Pre-existing issues:** Test failures unrelated to the migration sometimes hindered the automated process.

It's common for AI initiatives to encounter uncertainties like these – it can help to ensure developers are looped in at the right steps to mitigate risk and catch errors.

Google's approach demonstrates a strategic deployment of AI that accelerates software maintenance while maintaining high-quality standards. By focusing on routine but essential tasks like code migrations - the "low-hanging fruit" of software engineering - Google has created a structured initiative that delivers immediate value while minimizing risks.

This research shows how AI can transform engineering practices when deployed with careful planning and rigorous validation. The success metrics are compelling: with nearly 75% of code changes generated by AI and development time cut in half, Google has proven that targeted AI applications can dramatically improve developer productivity on routine tasks.

Key takeaway: Directors are the most prolific and diverse AI tool users

Directors reported the highest number of AI tools used, with 38% using OpenAI and 26% using Microsoft-related tools.

Their usage spans a wide spectrum of platforms, including enterprise-grade options (Watson, Amazon, Microsoft), innovation-focused tools (Claude, Replit, Kite), and developer-centric solutions (Tabnine, Codex, Deepcode). This suggests that directors are not only exploring tools directly but also orchestrating AI strategy, evaluating vendors, and guiding implementation across the organization.

Key takeaway: Software Developers lean toward execution-focused, lightweight AI tools

Developers report strong usage of OpenAI (40%) and Microsoft (18%), along with broader but smaller adoption of hands-on coding tools like Claude, Kite, Tabnine, Codex, and Deepcode.

They gravitate toward tools that integrate into coding workflows and boost individual productivity – indicating a preference for tactical AI augmentation over high-level orchestration. While developers have less tool diversity than directors, their choices skew toward tools that deliver immediate value during code creation or debugging.

Key takeaway: Managers use fewer AI tools overall, but sit at a strategic intersection

Managers report the fewest AI tools overall, with 17 OpenAI and 8 Microsoft tools, and minimal engagement with the longer tail of platforms. Their role likely involves overseeing adoption rather than frequent tool use. This positioning suggests that managers may lag in direct AI interaction, possibly highlighting a blind spot in how AI impacts team workflows.

How Meta is transforming software testing with AI-powered bug hunters

Meta has unveiled a powerful new tool that could redefine how software bugs are caught – Automated Compliance Hardening (ACH). This system uses large language models (LLMs) to generate targeted software tests, helping engineers catch meaningful bugs – like privacy issues or performance bottlenecks – before they go live.

Conventional testing tools often focus on achieving high code coverage, but that doesn't guarantee real-world bugs will be caught. Most coverage-driven tests can't detect nuanced or context-specific problems that truly matter to end users – like a privacy leak in a messaging app or a sudden spike in resource consumption.

That's where ACH comes in. ACH flips the traditional script by starting with what developers care about most: real bugs.

How ACH works

1. Plain-language prompts

Engineers describe the kinds of bugs they're worried about in simple language (e.g., "a leak of sensitive user data").

2. Smart mutations

ACH uses an LLM to mutate the existing code, simulating realistic bug scenarios that reflect the described problems.

3. Custom test generation

It then generates test cases specifically designed to trigger and detect those simulated faults.

This results in tests that are not only high quality but laser-focused on real engineering and business concerns.

Meta has already rolled out ACH across high-impact platforms like Facebook Feed, Instagram, Messenger, and WhatsApp. They've seen the following results:

- ACH analyzed over 10,000 Kotlin classes in Android apps.
- It generated over 9,000 mutants and 500+ privacy-focused test cases.
- In internal evaluations, 73% of the generated tests were accepted by engineers.
- Even more impressively, over one-third of those tests directly targeted privacy risks.

This isn't just about catching more bugs – it's about catching the right bugs. ACH enables:



Proactive quality

Engineers can define what “quality” means in their context and ACH creates tests to uphold that standard.



Faster iteration

Tests are generated automatically, freeing engineers to focus on fixing, not finding, issues.



Smarter safety

The use of LLMs means the system understands code context, producing more relevant and realistic test cases.

With ACH, Meta is showing that LLMs can go beyond chat interfaces – they can become deeply integrated tools in the software development lifecycle. As AI continues to evolve, intelligent systems like ACH will be key to building safer, faster, and more trustworthy software.



LinearB is an engineering productivity platform that helps enterprises define exactly how code is brought to production and maximize the efficiency of their engineering organizations. With full visibility and control over your team's operations, you can finally improve your developer efficiency, effectiveness, and experience.

[Book a demo](#)